Introduction à la programmation GPU avec CUDA

CY Tech 2025/2026
Ismail Bouajaja
ismail.bouajaja@gmail.com
Ingénieur et Doctorant en IA

Présence

Renseigner les informations et marquer la présence dans le google sheet suivant (le lien est partagé sur Discord):

Programmation GPU 2025/2026

Introduction à la programmation GPU avec CUDA

Quelles sont vos attentes ?



Sommaire du module

Jour 1 - Fondations du calcul GPU et CUDA

- L'ère des données massives et des calculs massifs
- CPU vs GPU vs accélérateurs (TPU, NPU, APU)
- Introduction à CUDA (modèle host/device, kernels, mémoire, threads)
- Implémentations de base (Python → C → CUDA)

Jour 2 - Optimisation des performances

- Architecture GPU (warps, SMs)
- Mémoire et accès (coalescing, shared memory)
- Techniques d’optimisation
- Études de cas

Jour 3 - Évaluation et mise en pratique

1. Introduction

1. Calculs massifs : L’ère des données massives et des calculs massifs

Nous vivons dans l’ère des données massives, où la capacité à analyser et manipuler de vastes ensembles de données en temps réel est essentielle.

Ces calculs s’effectuent à toutes les échelles, des smartphones aux datacenters.

1. Introduction - Calculs massifs

Question ouverte :

Quels sont quelques domaines où sont effectués les calculs de façon massive ?

1. Introduction - Calculs massifs

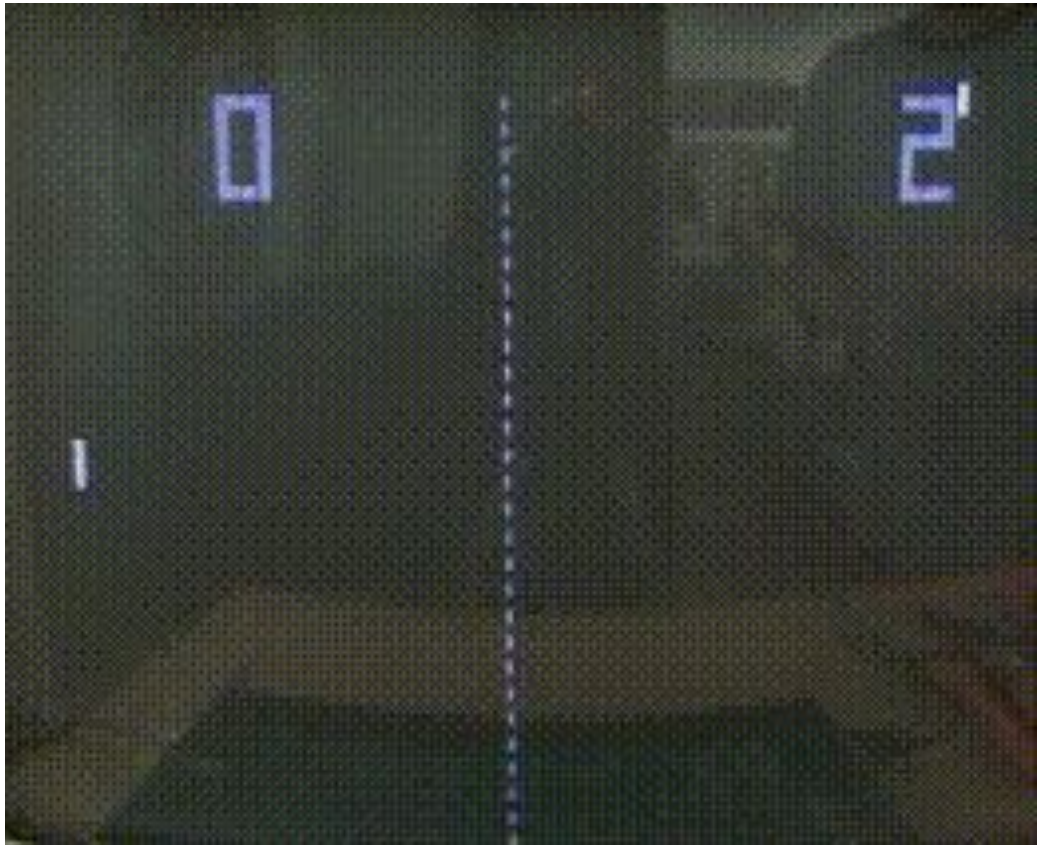


Jeux-vidéos

Contrairement aux films, les images affichées à l'écran dans un jeu vidéo sont générées en temps réel.

- Source :

Généré par DALL-E via ChatGPT



Source : By pong74ls, CC BY 3.0, https://commons.wikimedia.org/w/index.php?curid=79121297



Source : https://www.reddit.com/r/nvidia/comments/16tg3q0/cyberpunk_2077_has_the_best_graphics_ive_ever/#lightbox

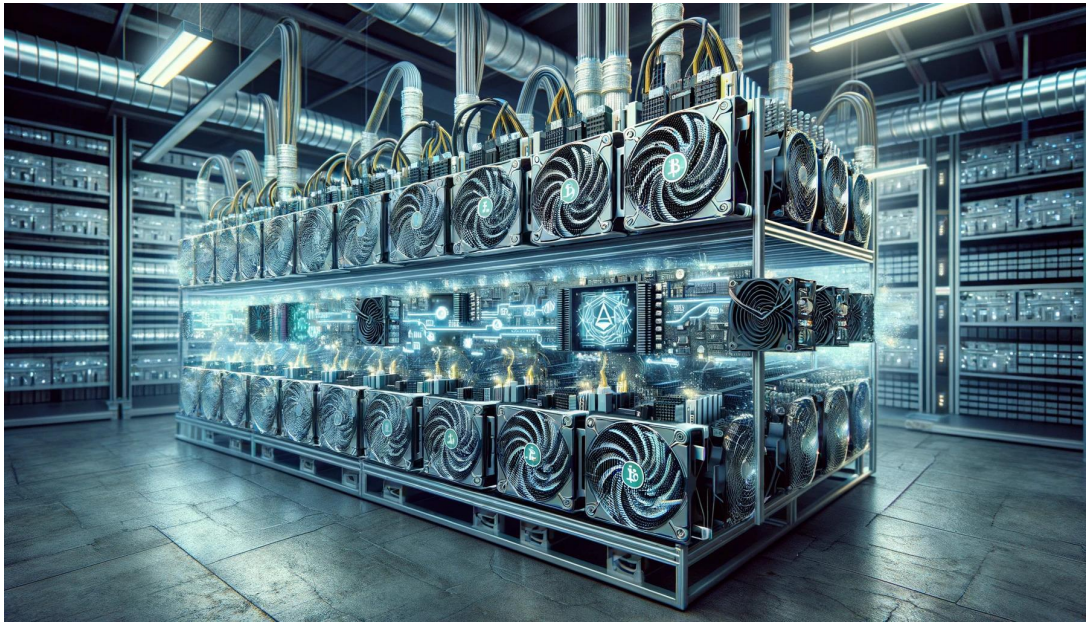


Source : https://wall.alphacoders.com/big.php?i=948667



Source : https://wall.alphacoders.com/big.php?i=948667

1. Introduction - Calculs massifs



Minage de Cryptomonnaies

Les GPUs sont essentiels pour résoudre les algorithmes cryptographiques du mining.

- Source :

Généré par DALL-E via ChatGPT

1. Introduction - Calculs massifs



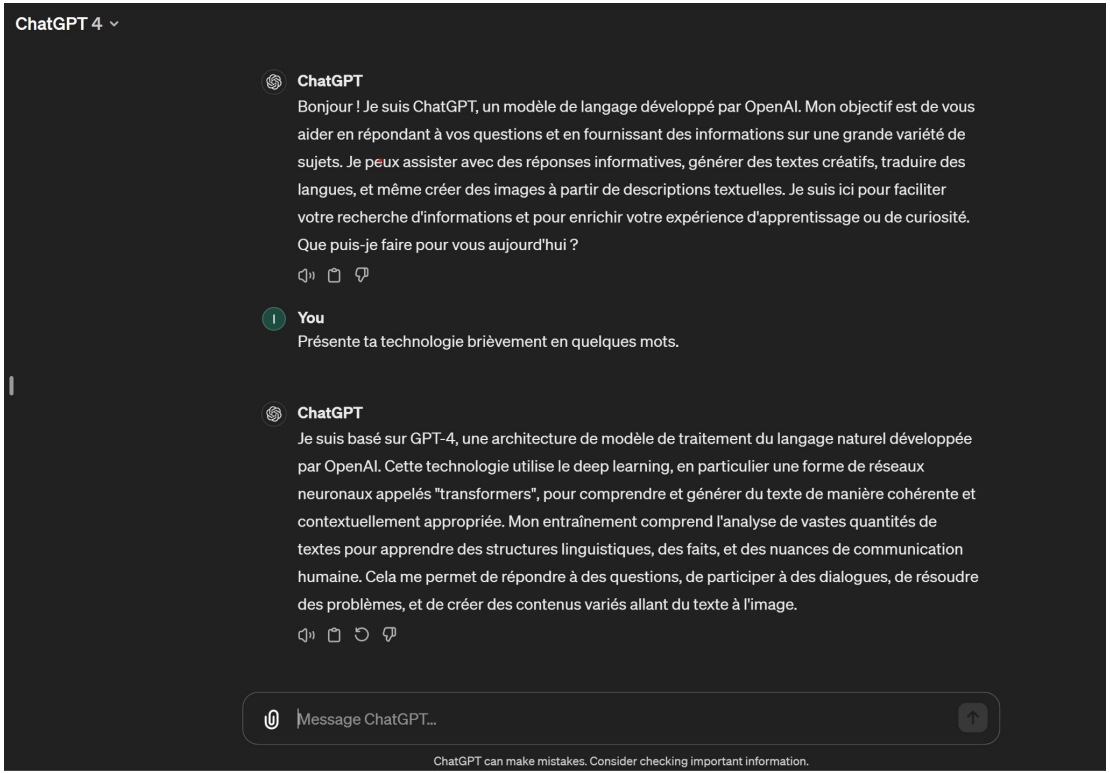
Multi-Agent Hide and Seek
by OpenAI

Des agents apprennent à jouer à cache-cache grâce à l'apprentissage par renforcement.

- Source :

https://www.youtube.com/watch?v=kopoLzvh5jY

1. Introduction - Calculs massifs



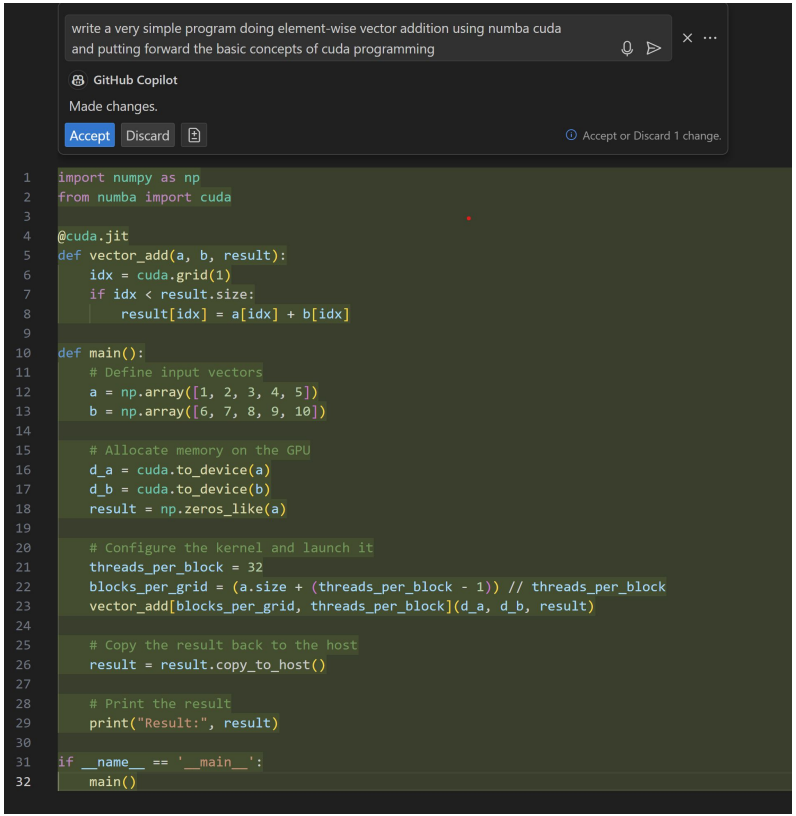
ChatGPT
by OpenAI

Le modèle de langage conversationnel développé par OpenAI.

- Source :

https://chat.openai.com/

1. Introduction - Calculs massifs



Github Copilot
by Github and OpenAI

GitHub Copilot est un outil de complétion de code développé par GitHub et OpenAI qui aide les programmeurs en complétant automatiquement le code.

- Source :

Capture d'écran dans VS Code

1. Introduction - Calculs massifs



SORA by OpenAI

- Prompt:

A movie trailer featuring the adventures of the 30 year old space man wearing a red wool knitted motorcycle helmet, blue sky, salt desert, cinematic style, shot on 35mm film, vivid colors.

- Source :

https://openai.com/sora

1. Introduction - Calculs massifs



All New Atlas
by Boston Dynamics

La robotique moderne, incarnée par des entreprises telles que Boston Dynamics, exploite les GPU pour construire des robots capables d'exécuter des tâches complexes avec une grande précision.

- Source :

https://www.youtube.com/watch?v=29ECwExc-_M

1. Introduction - Calculs massifs

- Intelligence Artificielle
- Robotique
- Médecine
- Traitement d’images et de vidéos
- Jeux vidéos
- Réseaux sociaux
- Météorologie
- Astronomie
- Finance
- …

1. Introduction

2. L’ère de la parallélisation : CPU vs GPU

Nous sommes entrés dans **l’ère de la parallélisation** :

- Augmentation massive des données
- Besoin de calculs rapides
- Limites du CPU (séquentiel)

➢ **Nécessité d’architectures parallèles (GPU)**

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un CPU ?



Source : https://www.freecodecamp.org/news/how-does-a-cpu-work/

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un CPU ?

“Un processeur (CPU) est un composant matériel qui constitue l'unité centrale de traitement d'un serveur. [...] Il extrait les instructions de la mémoire, exécute les tâches requises et renvoie les résultats en mémoire. Il gère toutes les tâches informatiques nécessaires au fonctionnement du système d'exploitation et des applications.”

Source : https://aws.amazon.com/fr/what-is/cpu/

2. Introduction - Accélération des calculs

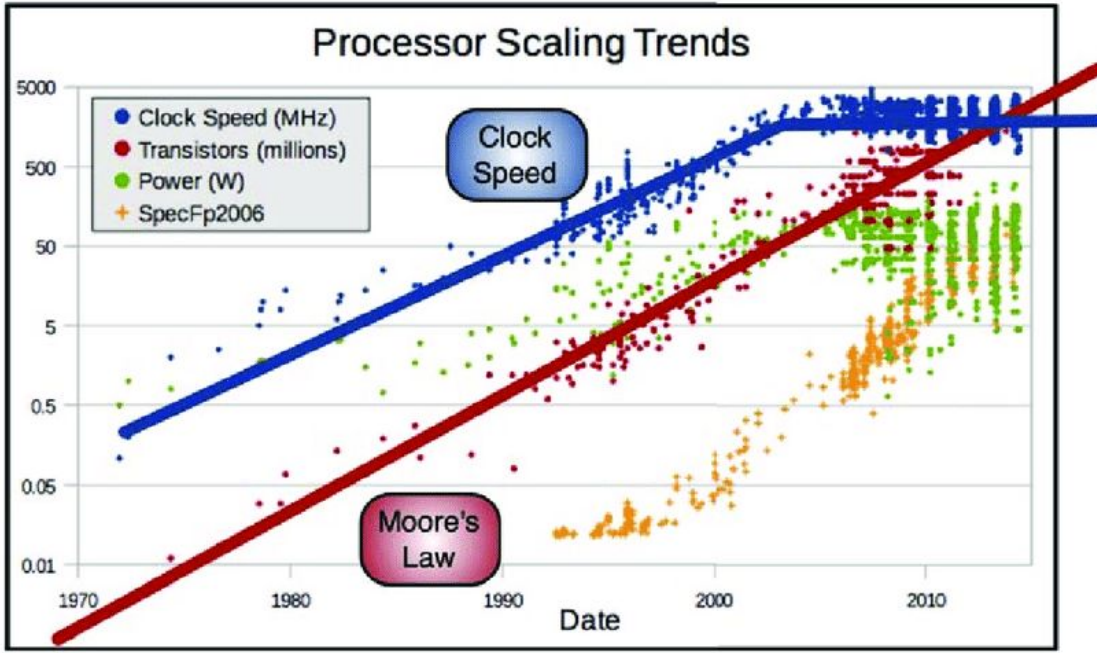
Question :

Qu’est-ce qu’un CPU ?

Typiquement, **un CPU excelle dans les tâches séquentielles**.

Les CPUs modernes sont multi-coeurs, proposant ainsi un certain niveau de parallélisme pour accélérer les systèmes informatiques davantage.

2. Introduction - Accélération des calculs



CPU transistor densities, clock speeds power and performance from 1970-2015. Courtesy of Charles Leggett, LBNL.

L'évolution des CPUs

La rapidité des CPUs stagnent depuis les années 2000 car nous atteignons la limite physique de réduction de la taille des transistors qui constituent les CPUs.

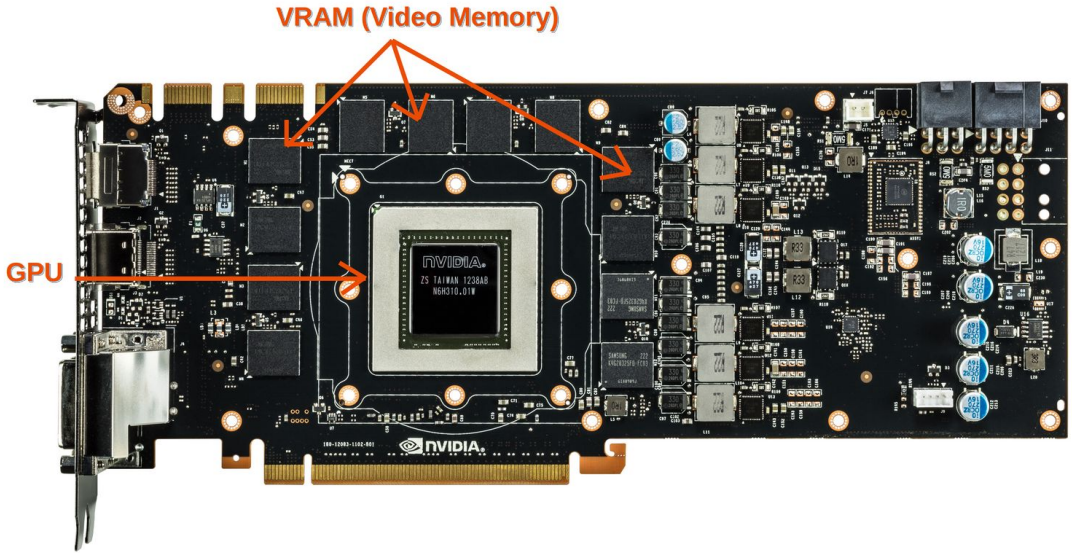
- Source :

Graeme A Stewart et al 2017 J. Phys.: Conf. Ser. 898 072013

2. Introduction - Accélération des calculs

Question :

Qu'est-ce qu'un GPU ?



Source : https://blog.paperspace.com/gpu-memory-bandwidth/

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un GPU ?

“Un processeur graphique (GPU) est un circuit électronique capable d'effectuer des calculs mathématiques à grande vitesse. [...] La conception d'un GPU lui permet d'effectuer la même opération sur plusieurs valeurs de données en parallèle. Cela augmente son efficacité de traitement pour de nombreuses tâches de calcul intensives.”

Source : https://aws.amazon.com/fr/what-is/gpu/

2. Introduction - Accélération des calculs

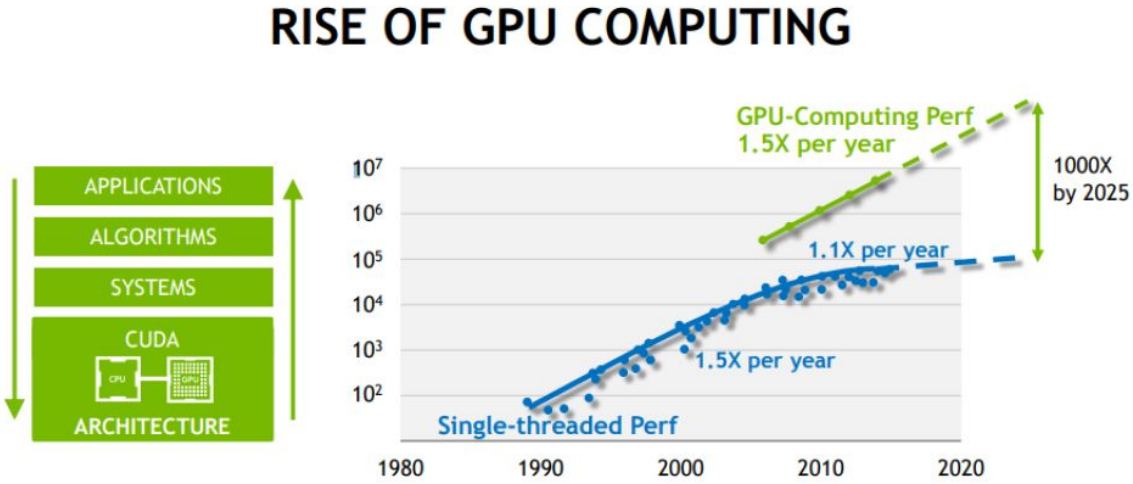
Question :

Qu’est-ce qu’un GPU ?

Typiquement, **un GPU excelle dans les tâches parallèles**.

Les GPUs ne sont pas bons pour les tâches non parallèles.

2. Introduction - Accélération des calculs



L’évolution des GPUs

L’aspect parallèle des GPUs leur permet de proposer des performances de plus en plus élevées chaque année et bien plus supérieures que les solutions non parallèles.

- Source :

Nvidia

https://punchcardinvestor.substack.com/p/nvidia-part-1-gpus-and-gaming

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un TPU ?



Source : https://cloud.google.com/blog/products/ai-machine-learning/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un TPU ?

“Les TPU (Tensor Processing Unit) sont des circuits intégrés spécifiques aux applications, conçus par Google pour accélérer les charges de travail de machine learning. [...] Les TPU sont conçus pour effectuer rapidement des opérations matricielles, ce qui les rend parfaits pour les charges de travail de machine learning.”

Source :
https://cloud.google.com/tpu/docs/system-architecture-tpu-vm?hl=fr

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un TPU ?

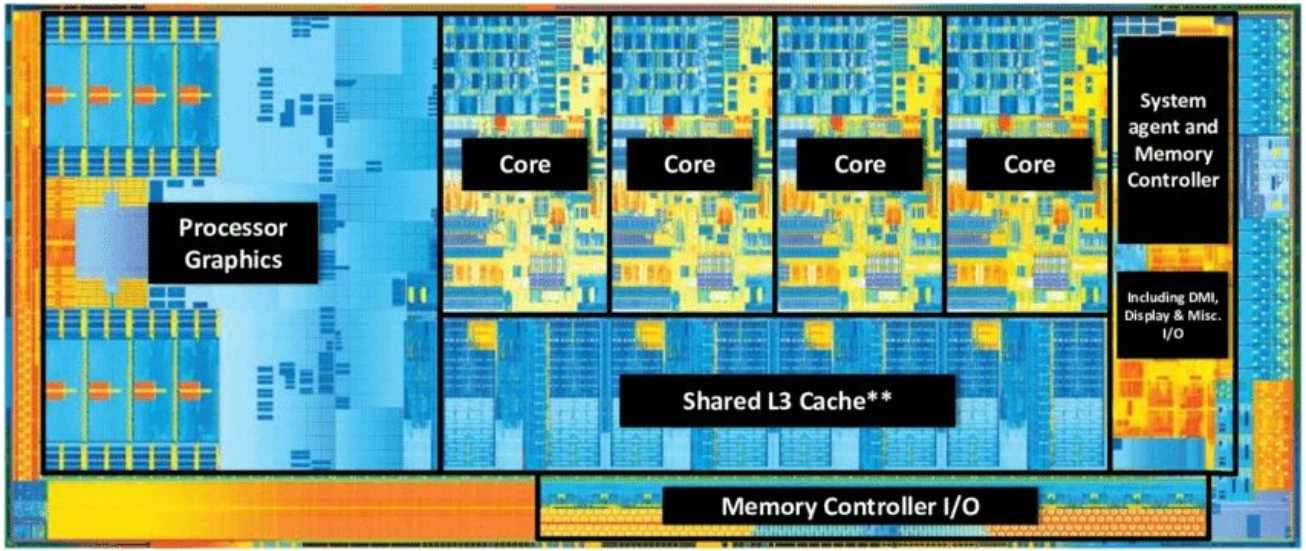
Typiquement, **un TPU excelle dans les opérations matricielles et tensorielles**.

Les TPUs sont principalement disponibles via GCP (Google Cloud Platform) et Google Colaboratory.

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un iGPU et un APU ?



Source : https://cpuninja.com/cpu-vs-apu/

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un iGPU et un APU ?

iGPU (integrated GPU)

- GPU intégré directement dans le CPU
- Partage la mémoire avec le CPU (RAM)
- **Moins performant qu’un GPU dédié mais plus efficient**

➢ Pour l'affichage de base, le décodage/encodage vidéo et les tâches légères GPU (petits jeux, accélération logicielle)

APU (Accelerated Processing Unit)

- Terme utilisé par AMD
- Désigne un CPU avec iGPU performant

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un NPU ?



Source : https://support.microsoft.com/fr-fr/windows/tout-sur-les-unit%C3%A9s-de-traitement-neuronal-npu-e77a5637-7705-4915-96c8-0c6a975f9db4

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un NPU ?

NPU (Neural Processing Unit)
Accélérateur spécialisé pour l’intelligence artificielle

- ✓ Optimisé pour les opérations d’inférence de réseaux de neurones (matmul, convolutions)
- ✓ Très efficace énergétiquement
- 🔻 Moins flexible qu’un GPU

➢ Utilisé pour de la reconnaissance vocale, de la vision (caméra, face ID), de l'IA embarquée

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un SoC ?



Source : https://arstechnica.com/gadgets/2025/12/the-npu-in-your-phone-keeps-improving-why-isnt-that-making-ai-better/

2. Introduction - Accélération des calculs

Question :

Qu’est-ce qu’un SoC ?

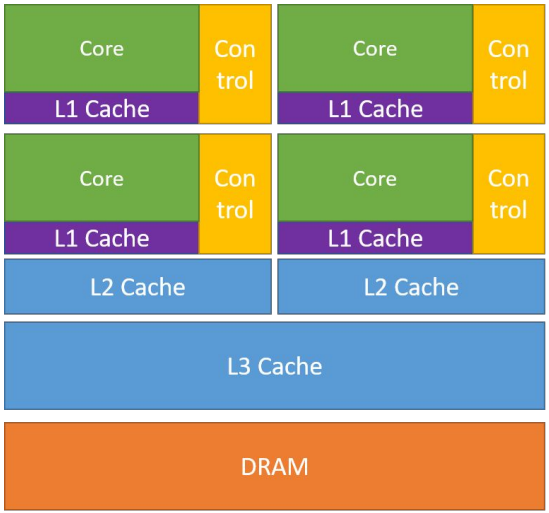
SoC (System on Chip)
Intègre plusieurs composants dans une seule puce :
CPU, GPU, NPU, contrôleur mémoire (souvent avec mémoire unifiée)

- ✓ Faible consommation
- ✓ Très bonne intégration
- ✓ Latence réduite (mémoire unifiée)
- 🔻 Moins flexible / évolutif
- 🔻 Pas remplaçable (tout est intégré)

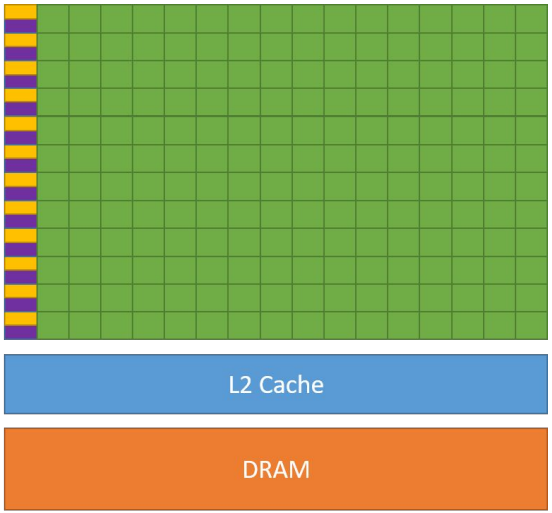
2. Introduction - Accélération des calculs

Type	Intel	AMD	NVIDIA	Apple	Google	Qualcomm
CPU	**Core Ultra**	**Ryzen / Ryzen AI**	❌	**M-series**	❌	Oryon
iGPU / APU	Intel Arc intégré / Xe	Radeon (APU)	❌	GPU intégré	❌	Adreno intégré
GPU	Arc	**Radeon**	**RTX**	GPU intégré	❌	❌
TPU	❌	❌	❌	❌	**TPU**	❌
NPU	**AI Boost**	**XDNA / Ryzen AI**	❌	**Neural Engine**	❌	**Hexagon**
SoC	❌	❌	Tegra / Jetson	**M-series**	Tensor (Pixel)	**Snapdragon**

2. Introduction - Accélération des calculs



CPU



GPU

CPU vs GPU

Le GPU mise sur un grand nombre de cœurs simples tandis que le CPU mise sur la rapidité de ses quelques coeurs.

- Source :

Nvidia

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#the-benefits-of-using-gpus

The GPU Devotes More Transistors to Data Processing

2. Introduction - Accélération des calculs

- Lorsqu'ils effectuent chacun une tâche, les CPUs optimisent le temps passé par opération (*latency*) alors que les GPUs optimisent le nombre d'opérations exécutées par unité de temps (*throughput*).
- Les GPUs ont un très grand nombre de coeurs (*cores*) simples en comparaison avec les CPUs.
- Les coeurs des GPUs ont une fréquence (*clock speed*) d'exécution inférieure à celle des CPUs.

Exemple :

- Les **GPUs** RTX 3080ti ont chacun **10240 coeurs** avec une fréquence variant entre **1.37GHz et 1.67Ghz**. (https://www.techpowerup.com/gpu-specs/geforce-rtx-3080-ti.c3735)
- Les **CPUs** Intel Core i9-12900K ont chacun **16 coeurs** avec une fréquence variant entre **3.2GHz et 5.2GHz**. (https://www.techpowerup.com/cpu-specs/core-i9-12900k.c2505)



2. Introduction - Accélération des calculs

Exercice :

Considérons 2 serveurs qui travaillent dans un même restaurant. Le premier serveur **S1** marche à **1 m/s** tandis que le deuxième serveur **S2** marche à **0.5 m/s**. Par ailleurs, le premier serveur **S1** ramène les plats **deux à la fois** tandis que le deuxième serveur **S2** a la possibilité de ramener **10 plats à la fois**.
Pour chacune des tables suivantes, donner la ***latency*** (le nombre de secondes par aller-retour complet), le ***throughput*** (nombre de plats servis par seconde) ainsi que la **durée totale** de chaque commande pour chaque serveur et déterminer **quel serveur a été le plus efficace**. On considère que les plats ont tous été livrés lorsque le serveur a effectué tous les allers-retours.

2. Introduction - Accélération des calculs

1. **table 1 :** située à 5 mètres des plats, a passé une commande de 20 plats.
   latency = ; throughput = ; durée totale = ; meilleur serveur = ;
2. **table 2 :** située à 10 mètres des plats, a passé une commande de 20 plats.
   latency = ; throughput = ; durée totale = ; meilleur serveur = ;
3. **table 3 :** située à 5 mètres des plats, a passé une commande de 4 plats.
   latency = ; throughput = ; durée totale = ; meilleur serveur = ;
4. **table 4 :** située à 10 mètres des plats, a passé une commande de 4 plats.
   latency = ; throughput = ; durée totale = ; meilleur serveur = ;
5. **table 5 :** située à 5 mètres des plates, a passé une commande de 2 plats.
   latency = ; throughput = ; durée totale = ; meilleur serveur = ;
6. **table 6 :** située à 10 mètres des plats, a passé une commande de 2 plats.
   latency = ; throughput = ; durée totale = ; meilleur serveur = ;

2. Introduction - Accélération des calculs

1. **table 1 :** située à 5 mètres des plats, a passé une commande de 20 plats.
   latency(1) = 10s ; throughput(1) = 0.2plat/seconde ; durée totale(1) = 100s;
   latency(2) = 20s ; throughput(2) = 0.5plat/seconde ; durée totale(2) = 40s;
   meilleur serveur = serveur 2;
2. **table 2 :** située à 10 mètres des plats, a passé une commande de 20 plats.
   latency(1) = 20s ; throughput(1) = 0.1plat/seconde ; durée totale(1) = 200s;
   latency(2) = 40s ; throughput(2) = 0.25plat/seconde ; durée totale(2) = 80s;
   meilleur serveur = serveur 2;

2. Introduction - Accélération des calculs

1. **table 3 :** située à 5 mètres des plats, a passé une commande de 4 plats.
   latency(1) = 10s ; throughput(1) = 0.2plat/seconde ; durée totale(1) = 20s;
   latency(2) = 20s ; throughput(2) = 0.2plat/seconde ; durée totale(2) = 20s;
   meilleur serveur = égalité;
2. **table 4 :** située à 10 mètres des plats, a passé une commande de 4 plats.
   latency(1) = 20s ; throughput(1) = 0.1plat/seconde ; durée totale(1) = 40s;
   latency(2) = 40s ; throughput(2) = 0.1plat/seconde ; durée totale(2) = 40s;
   meilleur serveur = égalité;

2. Introduction - Accélération des calculs

1. **table 5 :** située à 5 mètres des plates, a passé une commande de 2 plats.
   latency(1) = 10s ; throughput(1) = 0.2plat/seconde ; durée totale(1) = 10s;
   latency(2) = 20s ; throughput(2) = 0.1plat/seconde ; durée totale(2) = 20s;
   meilleur serveur = serveur 1;
2. **table 6 :** située à 10 mètres des plats, a passé une commande de 2 plats.
   latency(1) = 20s ; throughput(1) = 0.1plat/seconde ; durée totale(1) = 20s;
   latency(2) = 40s ; throughput(2) = 0.05plat/seconde ; durée totale(2) = 40s;
   meilleur serveur = serveur 1;

1. Introduction

3. Langages et programmation

Découvrons quelques langages à utiliser pour écrire des programmes à exécuter par un CPU et un GPU.

3. Introduction - Langages et programmation

```python
# !python arr_add.py

def init_data() -> tuple[list[int], list[int]]:
    size = int(10e6)
    arr1 = list(range(size))
    arr2 = list(range(size))
    return arr1, arr2

def compute(arr1: list[int], arr2: list[int]) -> list[int]:
    return [x + y for x, y in zip(arr1, arr2)]

def validate(arr1: list[int], arr2: list[int], result: list[int]) -> None:
    if result == [x + y for x, y in zip(arr1, arr2)]:
        print("SUCCESS : Test passed")
    else:
        print("FAILURE: Test failed")

def main():
    arr1, arr2 = init_data()
    result = compute(arr1, arr2)
    return arr1, arr2, result

if __name__ == "__main__":
    validate(*main())
```

Langage Python

- **Langage haut niveau (facile)**
- Langage interprété
- **Exécution lente en Python pur**
- Répandu dans le monde de l'IA grâce à ses librairies : scikit-learn, tensorflow, pytorch, numpy, scipy, ...

3. Introduction - Langages et programmation

```c
}

// Function to free allocated memory
void clean_up(int *arr1, int *arr2, int *results)
{
    free(arr1);
    free(arr2);
    free(results);
}

int main()
{
    int size = 1000000;
    int *arr1, *arr2, *results;

    // Initialize data
    init_data(&arr1, &arr2, &results, size);

    // Compute the results
    compute(arr1, arr2, results, size);

    // Test the results
    validate(arr1, arr2, results, size);

    // Clean up resources
    clean_up(arr1, arr2, results);

    return 0;
}
```

Langage C

- **Langage proche du matériel (bas niveau)**
- Langage compilé
- **Son exécution est très rapide**
- Plusieurs **librairies python sont implémentées en C** pour profiter de la rapidité du langage C, puis elles sont utilisées en python pour profiter de la simplicité du langage python

3. Introduction - Langages et programmation

```cuda
__global__ void d_compute(int *d_arr1, int *d_arr2, int *d_results, int size)
{
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    if (index < size)
        d_results[index] = d_arr1[index] + d_arr2[index];
}

void compute(int *arr1, int *arr2, int *results, int size)
{
    // Grid and block sizes
    int threadsPerBlock = 512;
    int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;
    // printf(
    //     "threadsPerBlock=%d, blocksPerGrid=%d\n",
    //     threadsPerBlock, blocksPerGrid);

    // Device memory
    int *d_arr1;
    int *d_arr2;
    int *d_results;

    // Allocate memory on device
    cudaMalloc((void **)&d_arr1, size * sizeof(int));
    cudaMalloc((void **)&d_arr2, size * sizeof(int));
    cudaMalloc((void **)&d_results, size * sizeof(int));

    // Copy data to device memory
    cudaMemcpy(d_arr1, arr1, size * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_arr2, arr2, size * sizeof(int), cudaMemcpyHostToDevice);

    // Compute on device
    d_compute<<<blocksPerGrid, threadsPerBlock>>>(d_arr1, d_arr2, d_results, size);
```

Langage CUDA

- Extension du langage C
- **Créé par Nvidia pour la programmation parallèle GPU**
- **Langage bas niveau (difficile)**
- Langage compilé
- **Son exécution est très rapide**

3. Introduction - Langages et programmation

Bibliothèque	Description
NumPy	Calcul matriciel sur **CPU**, opérations vectorisées rapides, base de **l'écosystème scientifique Python**
CuPy	**API similaire à NumPy** mais exécutée sur **GPU (CUDA)**, facilite la transition CPU → GPU
PyTorch	Tenseurs + **GPU** + autograd, **standard actuel pour le deep learning**
TensorFlow	Framework complet de deep learning, **optimisé pour CPU/GPU**, souvent utilisé en production
Numba	**Compile Python en code machine (CPU) ou GPU (CUDA)**, permet d'écrire du code performant sans quitter Python

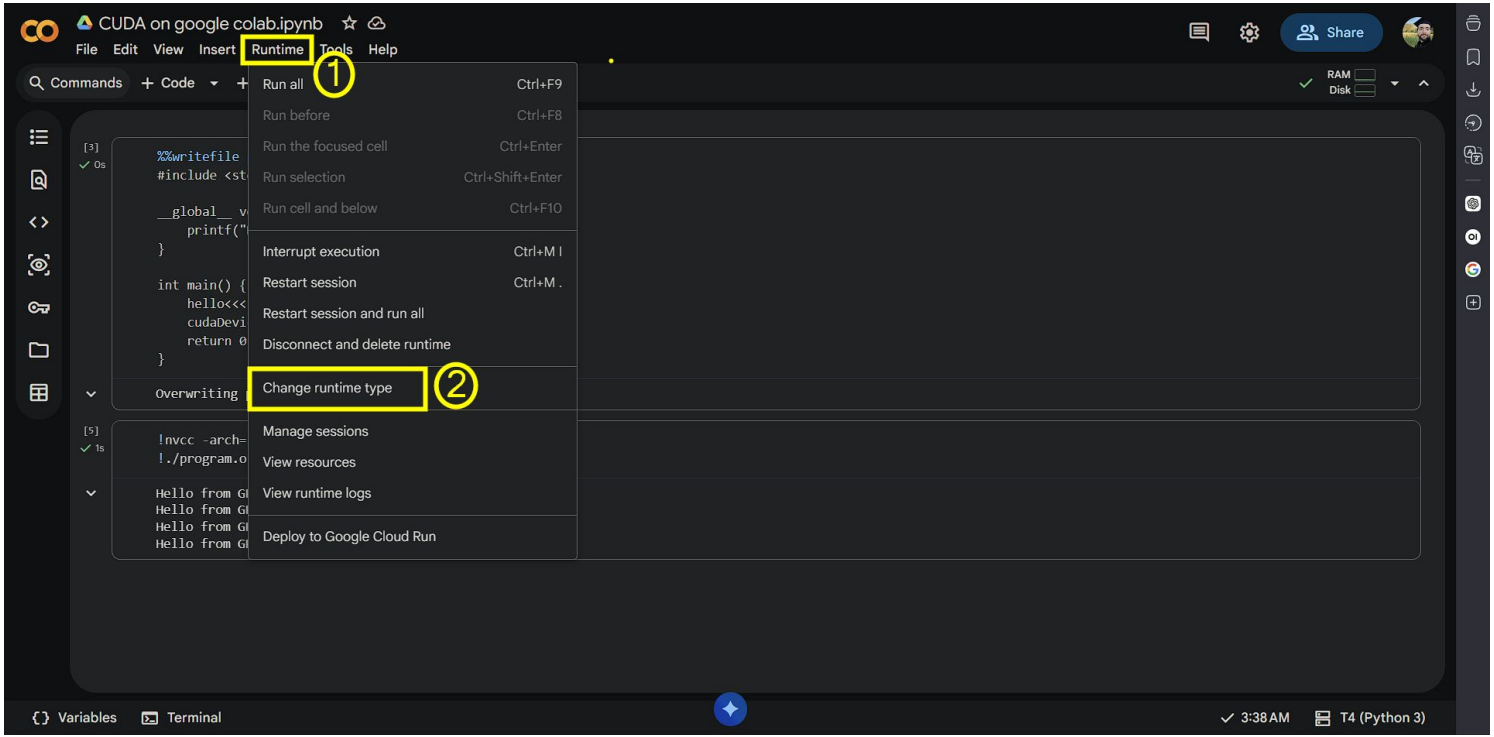
1. Introduction

4. Setup Google Colaboratory

Dans le cadre du cours, nous allons exécuter nos programmes dans Google Colaboratory qui met à disposition des GPUs gratuitement avec des environnements bien établis.

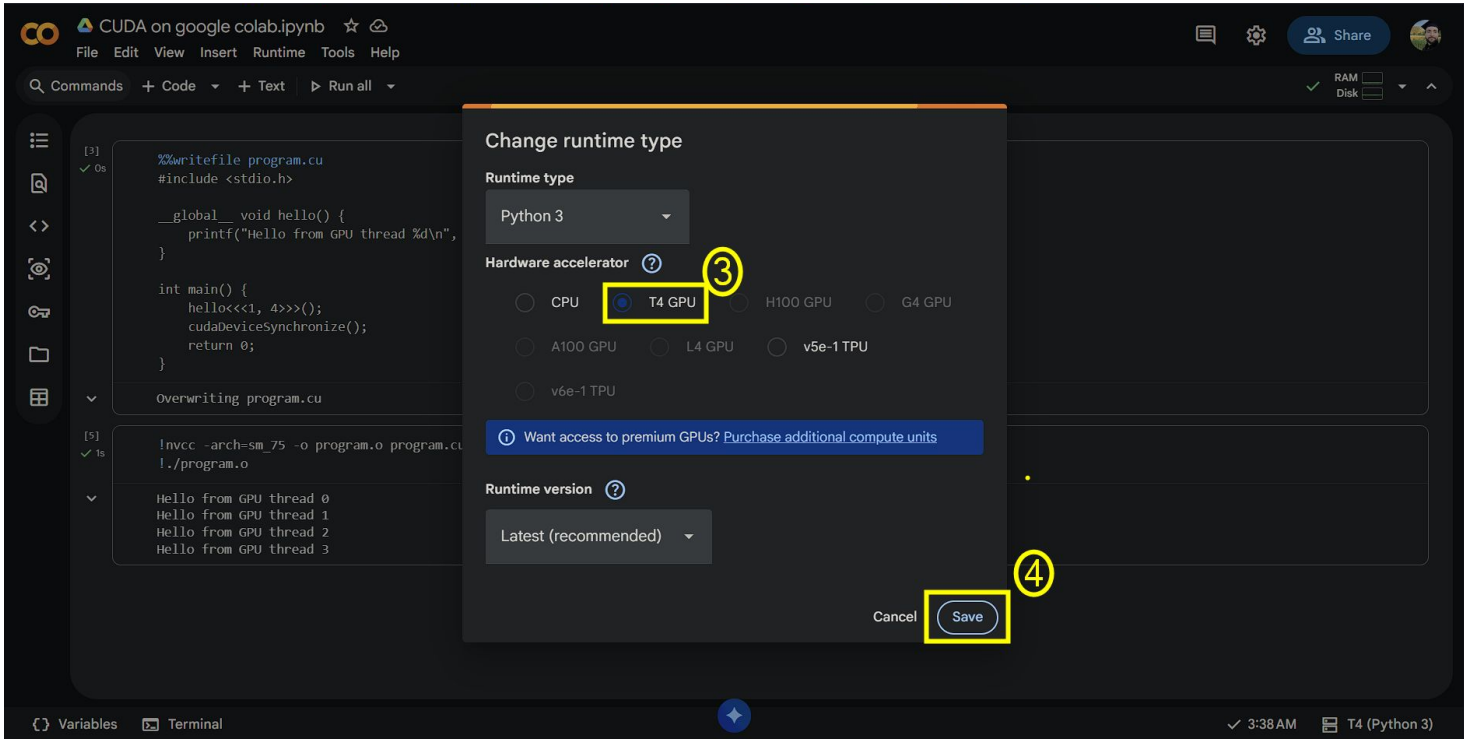
4. Introduction - Setup Google Colaboratory

étape 1 : choisir le runtime



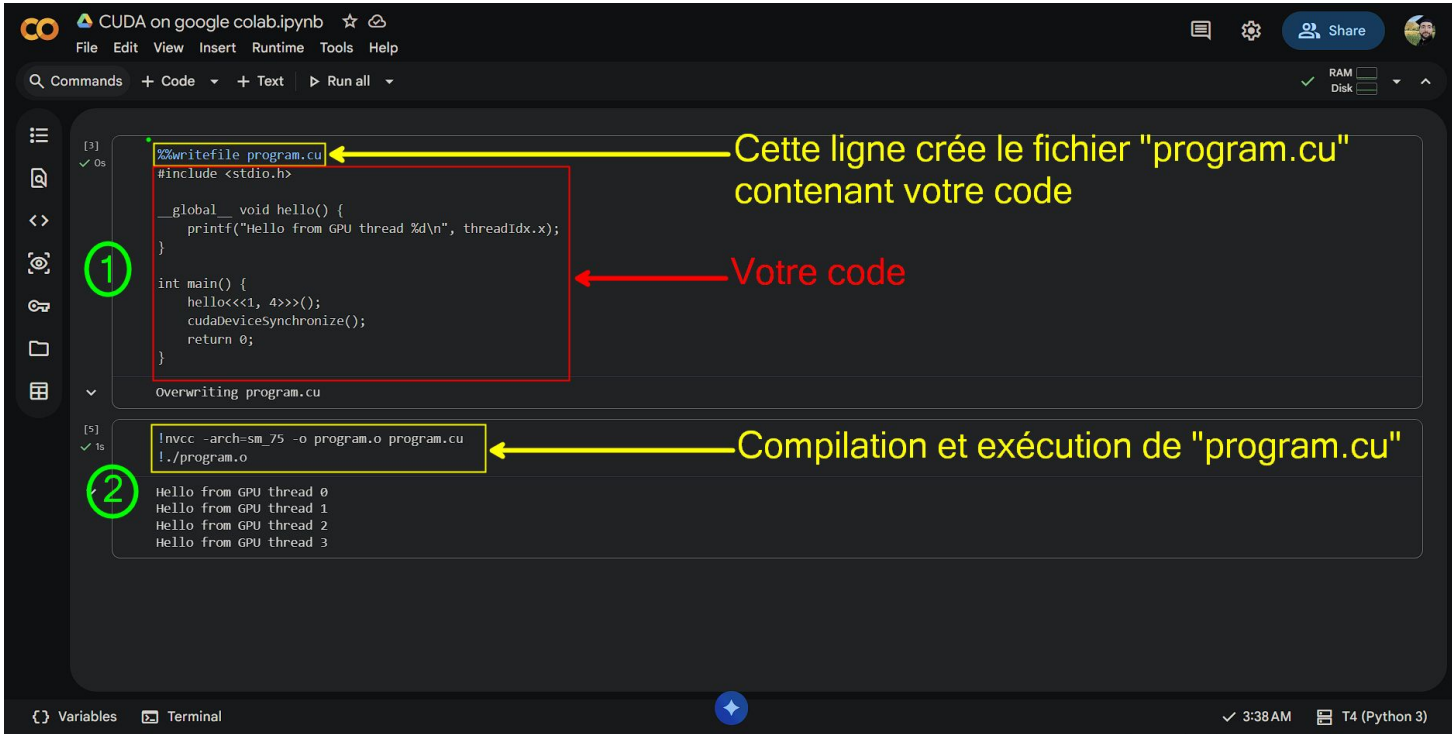
4. Introduction - Setup Google Colaboratory

étape 2 : choisir le GPU seulement si nécessaire, sinon CPU.



4. Introduction - Setup Google Colaboratory

étape 3 : écrire, compiler et exécuter le code



4. Introduction - Setup Google Colaboratory

Lien de l'exemple :
https://colab.research.google.com/drive/1cygVzVXgNjEeb3l_8HZlr7f3dUmuwjod?usp=sharing

1. Introduction

5. Programmation Python

Nous allons implémenter quelques algorithmes en utilisant le CPU seulement, en Python.

5. Introduction - Programmation Python

Ecrire les programmes suivants en Python :

1. Calculer le carré des éléments d'une liste d'entiers élément par élément.
2. Calculer la somme de deux listes d'entiers élément par élément.
3. Décaler tous les éléments d'un array d'entiers d'un cran vers la gauche (l'élément à la ième place est décalé à la place i-1), le premier élément est placé à la dernière place de l'array d'entiers.
4. Calculer la somme totale des éléments d'une liste d'entiers.
5. Calculer la transposée d'une matrice d'entiers 2D.
6. Calculer la multiplication matricielle de deux matrices 2D.

1. Introduction

5. Programmation C

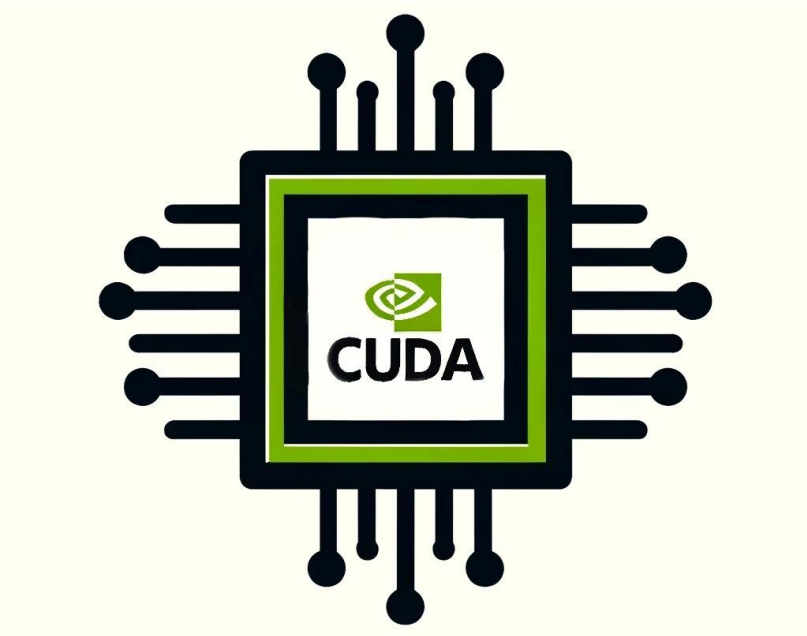
Nous allons implémenter quelques algorithmes en utilisant le CPU seulement, en C.

5. Introduction - Programmation C

Ecrire les programmes suivants en C :

1. Calculer le carré des éléments d'un array d'entiers élément par élément.
2. Calculer la somme de deux arrays d'entiers élément par élément.
3. Décaler tous les éléments d'un array d'entiers d'un cran vers la gauche (l'élément à la ième place est décalé à la place i-1), le premier élément est placé à la dernière place de l'array d'entiers.
4. Calculer la somme totale des éléments d'un array d'entiers.
5. Calculer la transposée d'une matrice d'entiers 2D.
6. Calculer la multiplication matricielle de deux matrices 2D.

2. Programmation CUDA



2. Programmation CUDA

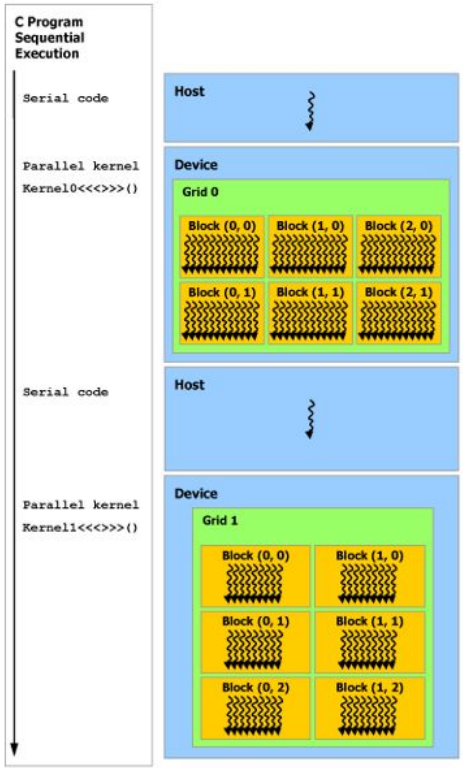
1. Programmation Hétérogène (*Heterogeneous Programming*)
2. Compilation et exécution
3. Hiérarchie des Threads (*Thread Hierarchy*)
4. Modèle de mémoire (*Memory Model*)
5. SMs (*Streaming Multiprocessors*) et extensibilité (*Scalability*)
6. Conditions de concurrence (*Race conditions*)
7. Parallélisation

2. Programmation CUDA

1. Programmation Hétérogène

La programmation CUDA est souvent désignée sous le terme de "programmation hétérogène" car elle implique l'utilisation coordonnée de plus d'un type de processeur, typiquement une unité de traitement central (CPU) et une unité de traitement graphique (GPU), pour effectuer des calculs.

1. Programmation CUDA - Programmation Hétérogène



Le *Host* (CPU) et le *Device* (GPU) sont physiquement séparés et chacun possède son propre espace mémoire DRAM, appelés *host memory* et *device memory* respectivement.

Le *Host* (CPU) exécute le programme principal et le *Device* (GPU) exécute les *kernels* (fonctions exécutées massivement en parallèle par le *Device* (GPU), lancées de manière asynchrone par le *Host* (CPU)).

Le *Host* (CPU) se charge :

1. d'allouer la mémoire nécessaire dans le *Device* (GPU)
2. de copier les données initiales du *Host* (CPU) vers le *Device* (GPU)
3. de lancer les *kernels* (fonctions) à exécuter par le *Device* (GPU)
4. de copier les données finales du *Device* (GPU) vers le *Host* (CPU)
5. de libérer la mémoire allouée dans le *Device* (GPU)

Source : Heterogenous Programming - https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

1. Programmation CUDA - Programmation Hétérogène

La définition d'un *kernel* est similaire à celle d'une fonction C ordinaire. Il suffit de rajouter le mot-clé (*declaration specifier*) **__global__** avant le type de retour de la fonction pour indiquer au compilateur qu'il s'agit d'une fonction qui sera lancée par le CPU pour être exécutée par le GPU.

Lancer un *kernel* se fait de façon similaire aux fonctions C en lui ajoutant **<<<...>>>** pour spécifier la configuration des threads *(voir Thread Hierarchy)*. Exemple : my_function<<<1, 1>>>(arg1, arg2)

Le langage CUDA nous met à disposition les fonctions suivantes :

- **cudaMalloc** : alloue de la mémoire dans le *Device* (GPU), similaire à la fonction C malloc
- **cudaMemcpy** : copie les données entre le *Host* (CPU) et le *Device* (GPU), similaire à la fonction C memcpy avec un paramètre supplémentaire qui prend une des valeurs suivantes :
  - **cudaMemcpyHostToDevice** pour les copies du *Host* (CPU) vers le *Device* (GPU)
  - **cudaMemcpyDeviceToHost** pour les copies du *Device* (GPU) vers le *Host* (CPU)
  - **cudaMemcpyDeviceToDevice** pour les copies du *Device* (GPU) vers le *Device* (GPU) *(peuvent être différents)*
- **cudaFree** : libère la mémoire allouée dans le *Device* (GPU), similaire à la fonction C free

2. Programmation CUDA

2. Compilation et exécution

CUDA est un langage compilé.

2. Programmation CUDA - Compilation et exécution

Pour compiler un programme CUDA dont le nom est par exemple *program.cu*, on utilise le compilateur **nvcc** via le terminal : nvcc program.cu.
nvcc admet quelques options comme *-o* pour préciser le nom du fichier compilé mais également *-gencode*, *-arch* et *-code* pour préciser quelles versions du fichier PTX et du fichier binaire générer.
Pour plus de détails, se référer à la documentation officielle :
https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#compilation-with-nvcc

2. Programmation CUDA - Compilation et exécution

Transformer les programmes C en CUDA : (sans parallélisation)

1. Calculer le carré des éléments d'un array d'entiers élément par élément.
2. Calculer la somme de deux arrays d'entiers élément par élément.
3. Décaler tous les éléments d'un array d'entiers d'un cran vers la gauche (l'élément à la ième place est décalé à la place i-1), le premier élément est placé à la dernière place de l'array d'entiers.
4. Calculer la somme totale des éléments d'un array d'entiers.
5. Calculer la transposée d'une matrice d'entiers 2D.
6. Calculer la multiplication matricielle de deux matrices 2D.

2. Programmation CUDA

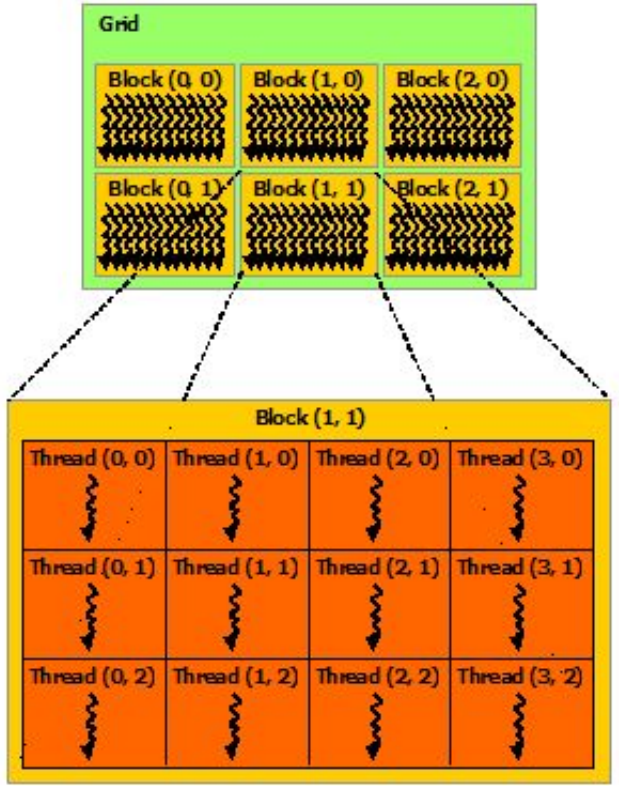
3. Hiérarchie des threads

La hiérarchie des threads dans CUDA est une abstraction qui permet de structurer et de gérer efficacement l'exécution parallèle des calculs sur le GPU. Cette abstraction aide les développeurs à concevoir des programmes qui exploitent le parallélisme à grande échelle du matériel de manière intuitive et organisée.

3. Programmation CUDA - Hiérarchie des threads

Les *kernels* sont toujours lancés en précisant la configuration de *grid* en fonction du type et de la dimensionnalité du problème à résoudre.

- *[compute] kernel* est une fonction lancée par le *Host* (CPU) et exécutée par le *Device* (GPU).
- *Thread* est une entité abstraite qui représente une exécution d'un *kernel*.
- *Block* est un espace 3D de *threads* (pouvant être réduit à 2 ou 1 dimension). (limité à 1024 threads par block dans les GPUs modernes)
- *Grid* est un espace 3D de *blocks* (pouvant être réduit à 2 ou 1 dimension).
- Chaque *thread* est identifié grâce à son index 3D du *thread* dans le *block et* à l'index 3D du *block* dans le *grid*.



3. Programmation CUDA - Hiérarchie des threads

Exercice :

1. Donner 6 configurations de *grid* possibles pour lancer un *kernel* avec 64 *threads*. (donner le nombre de *block* par dimension de la *grid* et le nombre de *threads* par dimension de *block*)
2. Quel nombre de dimensions (1, 2 ou 3 pour la *grid* et pour les *blocks*) choisir pour les problèmes suivants :
    a. addition de deux arrays élément par élément
    b. calcul de la transposée d'une matrice
    c. traitement d'une image en noir et blanc
    d. traitement d'une image rgba

3. Programmation CUDA - Hiérarchie des threads

Le nombre de *threads* par *block* et le nombre de *blocks* sont à spécifier lors de l'appel du *kernel* via la syntaxe **<<<**blocksPerGrid**,** threadsPerBlock**>>>**.

blocksPerGrid et threadsPerBlock peuvent être des entiers ou des structs de type **dim3**.
Un struct de type dim3 a trois attributs de type entier : **x**, **y** et **z**.
Ces trois attributs ont par défaut la valeur de 1.
Exemple : dim3 threadsPerBlock(16, 16); définit la variable threadsPerBlock avec x=16, y=16 et z=1.

L'implémentation du *kernel* se fait pour un seul *thread*. À l'intérieur de la définition du *kernel*, le programmeur CUDA a accès en particulier à 4 variables :

- **threadIdx** : un struct contenant la position du *thread* dans le *block* dans les 3 dimensions (**threadIdx.x**, **threadIdx.y**, **threadIdx.z**)
- **blockIdx** : un struct contenant la position du *block* dans la *grid* dans les 3 dimensions (**blockIdx.x**, **blockIdx.y**, **blockIdx.z**)
- **blockDim** : un struct contenant les 3 dimensions du *block*, c'est à dire le nombre de *threads* dans chacune des 3 dimensions (**blockDim.x**, **blockDim.y**, **blockDim.z**)
- **gridDim** : un struct contenant les 3 dimensions de la *grid*, c'est à dire le nombre de *blocks* dans chacune des 3 dimensions (**gridDim.x**, **gridDim.y**, **gridDim.z**)

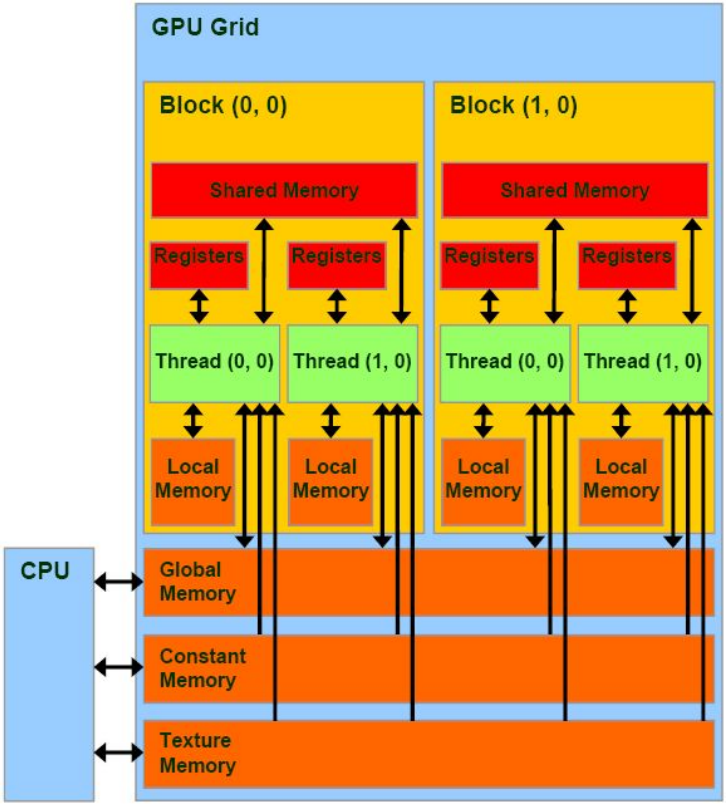
2. Programmation CUDA

4. Modèle de mémoire

Le modèle de mémoire de CUDA est conçu pour optimiser l'accès aux données sur les architectures de GPU. Il comprend plusieurs niveaux de mémoire tels que la mémoire globale, la mémoire partagée, et les registres, chacun offrant différents degrés de persistence et de vitesse d'accès pour supporter efficacement le parallélisme massif des threads.

4. Programmation CUDA - Modèle de mémoire

Register : mémoire de petite taille mais très rapide et propre à chaque *thread*.
Local Memory : mémoire lente (car stockée en global memory) mais également propre à chaque *thread,* utilisée lorsque le *register* n'est pas de taille suffisante.
Shared Memory : mémoire de petite taille mais très rapide partagée entre tous les *threads* d'un même *block*.
Global Memory : mémoire de grande taille mais très lente partagée entre tous les *threads* de tous les *blocks*. Accessible par le *Host* (CPU).
Constant Memory : mémoire se situant physiquement au même niveau que la *global memory* mais en lecture seule, ce qui la rend plus rapide grâce au cache.



4. Programmation CUDA - Modèle de mémoire

- *Global memory* : La mémoire qui est allouée sur le *Device* (GPU) par le *Host* (CPU) est placée dans la *global memory* du *Device* (GPU). Cette mémoire allouée est disponible pour tous les *threads* de tous les *blocks* et de tous les *kernels* tant qu'elle n'a pas été libérée par le *Host* (CPU).
- *Registers* et *Local memory* : Les variables déclarées au sein d'un *kernel* sont par défaut placées dans les *registers*. À défaut, elles sont alors placées dans la *local memory*. Dans les deux cas, chaque *thread* a sa propre version de ces variables, elles sont donc privées au *thread* et leur mémoire est libérée à la fin de la durée de vie du *thread* correspondant.
- *Shared memory* : Pour utiliser une variable au niveau de la *shared memory*, il faut la déclarer à l'intérieur du *kernel* en la précédant par le mot clé **__shared__**. Cette ligne de code est exécutée une seule fois par *block*. Chaque *block* a sa propre version de cette variable, elle est donc privée au *block*. Tous les *threads* du même *block* ont accès à cette variable et sa mémoire est libérée à la fin de la durée de vie du *block*, c'est-à-dire à la fin de la durée de vie de tous les *threads* du *block*.

2. Programmation CUDA

5. SMs et extensibilité

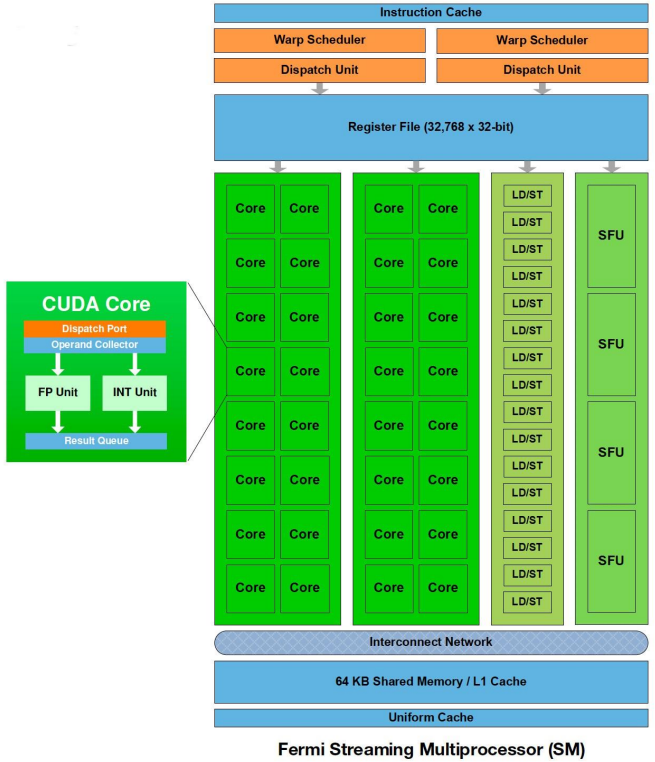
Les multiprocesseurs de flux ou Streaming Multiprocessors (SMs) dans l'architecture CUDA sont des unités de calcul centralisées sur les GPU NVIDIA qui exécutent simultanément des blocs de threads. Chaque SM est capable de gérer plusieurs threads à la fois, optimisant ainsi les performances parallèles et l'efficacité énergétique du GPU.

5. Programmation CUDA - SMs et extensibilité

Un GPU est constitué de plusieurs *SMs* (*Streaming Multiprocessors*).

Chaque *SM* contient :

- plusieurs *[execution] cores*
- des caches (*Shared Memory*, ...)
- des registres
- ordonnanceurs/planificateurs pour warps (*schedulers for warps*)

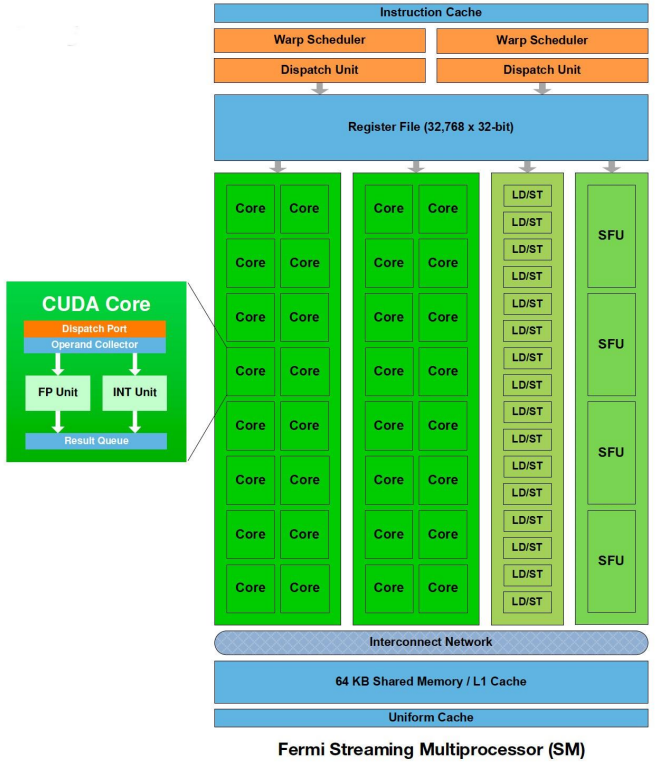


5. Programmation CUDA - SMs et extensibilité

Tous les *threads* d'un même *block* s'exécutent dans un même *SM*.

Plusieurs *blocks* de *threads* peuvent s'exécuter simultanément dans un même *SM*.

Un *block* de *threads* est planifié dans un *SM* dès que les ressources nécessaires pour son exécution sont disponibles. *(nombre de coeurs requis, mémoire requise, ...)*



5. Programmation CUDA - SMs et extensibilité

Question :

Considérons une carte graphique dont les *SMs* ont la capacité maximale de lancer 1536 *threads* simultanément et un programme nécessitant 4096 threads. Laquelle(lesquelles) des configurations de *grid* suivantes est(sont) correcte(s) ?

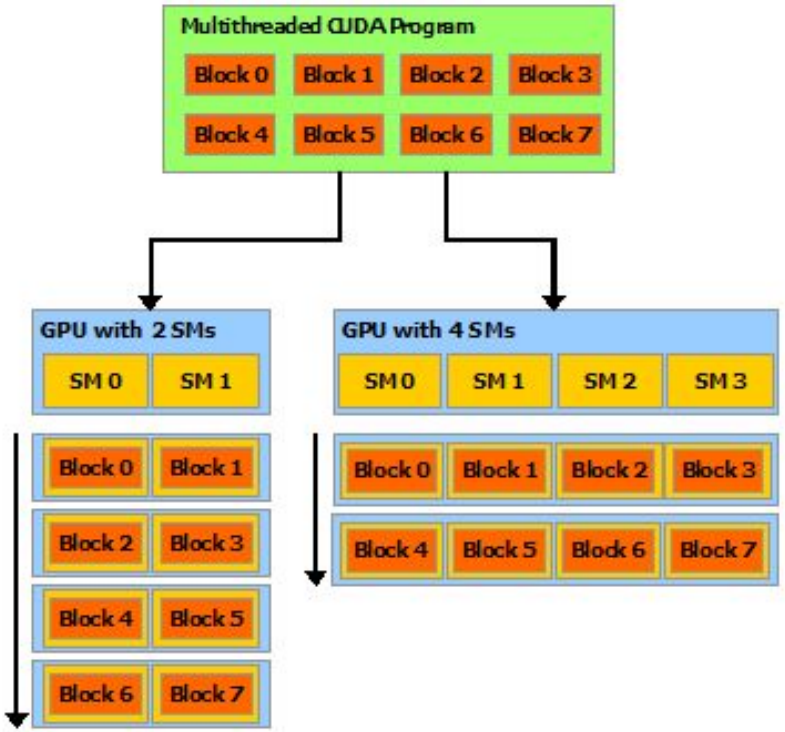
1. config 1 : 4 *blocks* par *grid* et 1024 *threads* par *block*.
2. config 2 : 6 *blocks* par *grid* et 512 *threads* par *block*.
3. config 3 : 1 *block* par *grid* et 4096 *threads* par *block*.
4. config 4 : 10 *blocks* par *grid* et 512 *threads* par *block*.

5. Programmation CUDA - SMs et extensibilité

Le système de *grid* offre cet aspect très important d'extensibilité (*scalability*) des GPUs.

Le programme fonctionne en terme de *blocks* indépendamment du nombre de *SMs* effectivement disponibles dans le(s) GPU(s) utilisé (s).

Le même programme peut être exécuté dans différents GPUs de différentes tailles.



2. Programmation CUDA

6. Conditions de concurrence

Les conditions de concurrence dans CUDA surviennent lorsque plusieurs threads tentent simultanément de lire ou d'écrire dans une même zone de mémoire sans synchronisation adéquate, pouvant mener à des incohérences et des erreurs dans les résultats. Ces situations exigent des mécanismes de synchronisation précis pour garantir l'intégrité des données.

6. Programmation CUDA - Conditions de concurrence

Trois options pour éviter les conditions de concurrence (*race conditions*) :

- Repenser son algorithme pour éviter les écritures/lectures simultanées et problématiques d'adresses mémoires par différents threads
- **__syncthreads();** : Cette fonction permet de synchroniser tous les *threads* d'un même *block*
- *Atomics* : Ce sont des fonctions mises à disposition par CUDA qui garantissent l'atomicité des opérations effectuées dans une adresse mémoire spécifique.

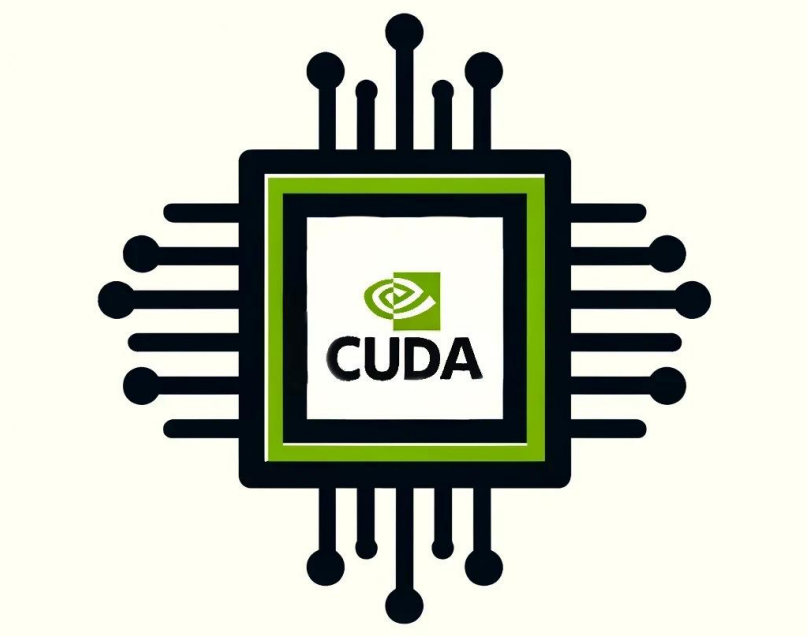
Exemple :
On considère un *kernel* lancée avec 64 *threads*.
Une variable *(int*) d* est placée dans la *global memory* (partagée donc entre tous les 64 *threads* du *kernel*).
En initialisant la valeur de **d* à *0* puis en exécutant atomicAdd(d, 1) pour tous les 64 *threads*, on peut être certain que la nouvelle valeur de **d* correspondra bien au nombre de *threads*, *c'est-à-dire *d == 64.*

7. Programmation CUDA - Parallélisation

Paralléliser les programmes CUDA :

1. Calculer le carré des éléments d'un array d'entiers élément par élément.
2. Calculer la somme de deux arrays d'entiers élément par élément.
3. Décaler tous les éléments d'un array d'entiers d'un cran vers la gauche (l'élément à la ième place est décalé à la place i-1), le premier élément est placé à la dernière place de l'array d'entiers.
4. Calculer la somme totale des éléments d'un array d'entiers.
5. Calculer la transposée d'une matrice d'entiers 2D.
6. Calculer la multiplication matricielle de deux matrices 2D.

3. Optimisations



L’optimisation d’un programme CUDA dépend du problème traité, de l’algorithme choisi et du GPU lui-même mais il existe des bonnes pratiques à connaître pour obtenir des bonnes performances.

1. Optimisations - WARP

Lors de l'exécution d'un *kernel*, les *threads* ne sont pas indépendants les uns des autres. Ils sont regroupés en groupe de 32 *threads* qu'on appelle des *warp*. Tous les *threads* d'un même *warp* effectuent exactement la même instruction au même instant.
Dans le cadre de l'optimisation des programmes CUDA, il faut prendre en compte ces *warps*. Notamment, il faut éviter les conditions (if...) que seule une partie des *threads* d'un même *warp* exécutera car le reste des *threads* sera inactif durant cette exécution. On appelle cela ***warp divergence***.

2. Optimisations - Mémoire

En général, les performances du programme CUDA dépendent de la façon dont est utilisée la mémoire. Pour obtenir de bonnes performances, il faut considérer les points suivants :

- Lorsqu'il faut accéder plusieurs fois à des données, il vaut mieux les copier dans la *local memory* ou la *shared memory* si ces derniers sont dans la *global memory* étant donné que **la *local memory* et la *shared memory* sont beaucoup plus rapides que la *global memory***.
- Il est préférable d'**accéder à des tranches continues de la *global memory* (*coalesced access*) par des *threads* adjacents** plutôt que d'accéder à des adresses dispersées. Cela permet de récupérer plusieurs adresses par requête, car sinon, dans le pire des cas, il y aura autant de requêtes que d'adresses.
- Il est également préférable d'**accéder à des adresses continues de la *shared memory*** car cela permet d'**éviter au maximum les *bank conflicts***. La *shared memory* est divisée en plusieurs modules appelés *banks*. Il est possible de récupérer des données en même temps si chaque *thread* récupère une donnée d'une *bank* différente des autres ou bien lorsque la donnée récupérée par tous les *threads* est la même. Si deux *threads* ou plus veulent récupérer une donnée différente dans la même *bank* alors ce ne sera pas réalisé en même temps. C'est ce qu'on appelle ***bank conflict***.

3. Optimisations - Hardware

Pour optimiser davantage les performances de son programme CUDA, il faut que ce dernier soit adapté au *hardware* (GPU) sur lequel il sera exécuté afin de pouvoir par exemple lancer un maximum de *blocks* en parallèle. Il faut donc prendre en compte en particulier :

- Le nombre maximal de *threads* par *SM*
- Le nombre maximal de *blocks* par *SM*
- Le nombre total de *SMs*
- La taille maximale de *shared memory* par *SM*
- Le nombre maximal de *registers* par *SM*
- ...D'autres facteurs qui peuvent influencer les performances sont disponibles grâce à **DeviceQuery**

4. Optimisations - Implémentations

Optimiser les programmes suivants en CUDA :

1. Calculer la transposée d’une matrice d’entiers 2D.
2. Calculer la multiplication matricielle de deux matrices 2D.
3. Calculer la somme totale des éléments d’une liste d’entiers.